

《基于 MindSpore 的红酒分类 实验》



华为技术有限公司

版权所有 © 华为技术有限公司 2024 保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



和其他华为商标均为华为技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

华为技术有限公司		
地址：	深圳市龙岗区坂田华为总部办公楼	邮编：518129
网址：	http://e.huawei.com	



目录

1 红酒分类实验	2
1.1 实验介绍	2
1.1.1 简介	2
1.1.2 实验目的	2
1.2 实验环境搭建	2
1.3 实验过程	3
1.3.1 数据准备	3
1.3.2 模型构建--计算距离	6
1.3.3 模型预测	7
1.4 实验小结	7

1 红酒分类实验

1.1 实验介绍

1.1.1 简介

K 近邻算法 (K-Nearest-Neighbor, KNN) 是一种用于分类和回归的非参数统计方法，是机器学习最基础的算法之一。KNN 也是有监督学习中的分类算法。KNN 的三个基本要素：

- K 值，一个样本的分类是由 K 个邻居的“多数表决”确定的。K 值越小，容易受噪声影响，反之，会使类别之间的界限变得模糊。
- 距离度量，反映了特征空间中两个样本间的相似度，距离越小，越相似。常用的有 L_p 距离 ($p=2$ 时，即为欧式距离)、曼哈顿距离、海明距离等。
- 分类决策规则，通常是多数表决，或者基于距离加权的多数表决 (权值与距离成反比)。

本实验主要介绍使用 MindSpore 在部分 wine 数据集上进行 KNN 实验。

1.1.2 实验目的

- 了解 KNN 的基本概念；
- 了解如何使用 MindSpore 进行 KNN 实验；

1.2 实验环境搭建

预备知识：

- 熟练使用 Python。
- 具备一定的机器学习理论知识，如 KNN、无监督学习、欧式距离等。

实验环境：

- MindSpore 2.2 (MindSpore 版本会定期更新，本指导也会定期刷新，与版本配套)；
- 本案例支持 win_x86 和 Linux 系统，CPU/GPU/Ascend 均可运行。

本实验在华为云 ModelArts 上进行，开发环境搭建可参考如下手册：ModelArts 云环境环境搭建指南。



华为云ModelArts
环境搭建手册.docx

1.3 实验过程

1.3.1 数据准备

步骤 1 下载数据

Wine 数据集是模式识别最著名的数据集之一，Wine 数据集的官网：[Wine Data Set](<http://archive.ics.uci.edu/ml/datasets/Wine>)。这些数据是对来自意大利同一地区但来自三个不同品种的葡萄酒进行化学分析的结果。数据集分析了三种葡萄酒中每种所含 13 种成分的量。这些 13 种属性如下表所示：

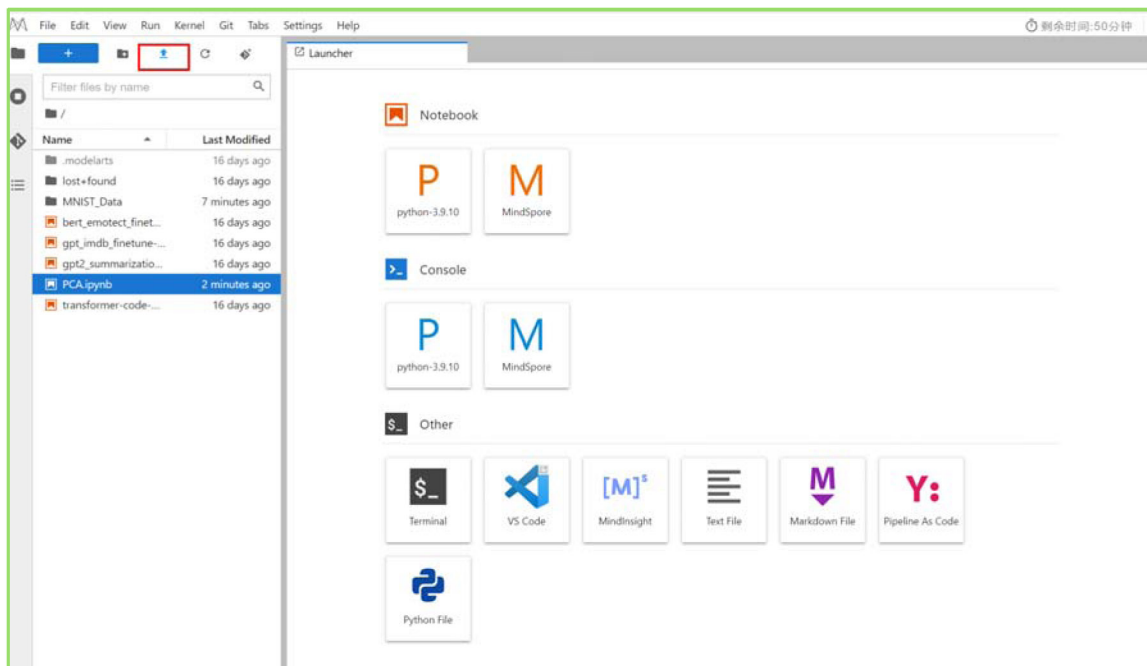
序号	成分（英文）	成分（中文）
1	Alcohol	酒精
2	Malic acid	苹果酸
3	Ash	灰
4	Alcalinity of ash	灰的碱度
5	Magnesium	镁
6	Total phenols	总酚
7	Flavanoids	类黄酮
8	Nonflavanoid phenols	非黄酮酚
9	Proanthocyanins	原花青素
10	Color intensity	色彩强度
11	Hue	色调
12	OD ₂₈₀ /OD ₃₁₅ of diluted wines	稀释酒的OD ₂₈₀ /OD ₃₁₅
13	Proline	脯氨酸

表1-1 葡萄酒成分表

下载 wine.data 数据文件，下载链接：

ascend-professional-construction-dataset.obs.cn-north-4.myhuaweicloud.com

实验开始前，在云环境中，请通过下图所示的上传按钮，将下载的数据上传至 ModelArts 实验平台。



步骤 2 导入 MindSpore 模块和辅助模块

```
#%matplotlib inline
import os
import csv
import numpy as np
import matplotlib.pyplot as plt

import mindspore as ms
from mindspore import nn, ops

ms.set_context(device_target="Ascend")
```

步骤 3 读取 Wine 数据集 wine.data，并查看部分数据。

输入：

```
with open('wine.data') as csv_file:
    data = list(csv.reader(csv_file, delimiter=','))
print(data[56:62]+data[130:133]) # 打印部分数据
```

输出：

```
[[1, '14.22', '1.7', '2.3', '16.3', '118', '3.2', '3', '.26', '2.03', '6.38', '.94', '3.31', '970'], [1, '13.29', '1.97', '2.68', '16.8', '102', '3', '3.23', '.31', '1.66', '6', '1.07', '2.84', '1270'], [1, '13.72', '1.43', '2.5', '16.7', '108', '3.4', '3.67', '.19', '2.04', '6.8', '.89', '2.87', '1285'], [2, '12.37', '.94', '1.36', '10.6', '88', '1.98', '.57', '.28', '.42', '1.95', '1.05', '1.82', '520'], [2, '12.33', '1.1', '2.28', '16', '101', '2.05', '1.09', '.63', '.41', '3.27', '1.25', '1.67', '680'], [2, '12.64', '1.36', '2.02', '16.8', '100', '2.02', '1.41', '.53', '.62', '5.75', '.98', '1.59', '450'], [3, '12.86', '1.35', '2.32', '18', '122', '1.51', '1.25', '.21', '.94', '4.1', '.76', '1.29', '630'], [3, '12.88', '2.99', '2.4', '20', '104', '1.3', '1.22', '.24', '.83', '5.4', '.74', '1.42', '530'], [3, '12.81', '2.31', '2.4', '24', '98', '1.15', '1.09', '.27', '.83', '5.7', '.66', '1.36', '560']]
```

步骤 4 取三类样本（共 178 条），将数据集的 13 个属性作为自变量 X。将数据集的 3 个类别作为因变量 Y。

```
X = np.array([[float(x) for x in s[1:]] for s in data[:178]], np.float32)
Y = np.array([s[0] for s in data[:178]], np.int32)
```

步骤 5 取样本的某两个属性进行 2 维可视化，可以看到在某两个属性上样本的分布情况以及可分性。

```
attrs = ['Alcohol', 'Malic acid', 'Ash', 'Alcalinity of ash', 'Magnesium', 'Total phenols',
         'Flavanoids', 'Nonflavanoid phenols', 'Proanthocyanins', 'Color intensity', 'Hue',
         'OD280/OD315 of diluted wines', 'Proline']
plt.figure(figsize=(10, 8))
for i in range(0, 4):
    plt.subplot(2, 2, i+1)
    a1, a2 = 2 * i, 2 * i + 1
    plt.scatter(X[:, a1], X[:, a2], label='1')
    plt.scatter(X[59:130, a1], X[59:130, a2], label='2')
    plt.scatter(X[130:, a1], X[130:, a2], label='3')
    plt.xlabel(attrs[a1])
    plt.ylabel(attrs[a2])
    plt.legend()
plt.show()
```

输出：

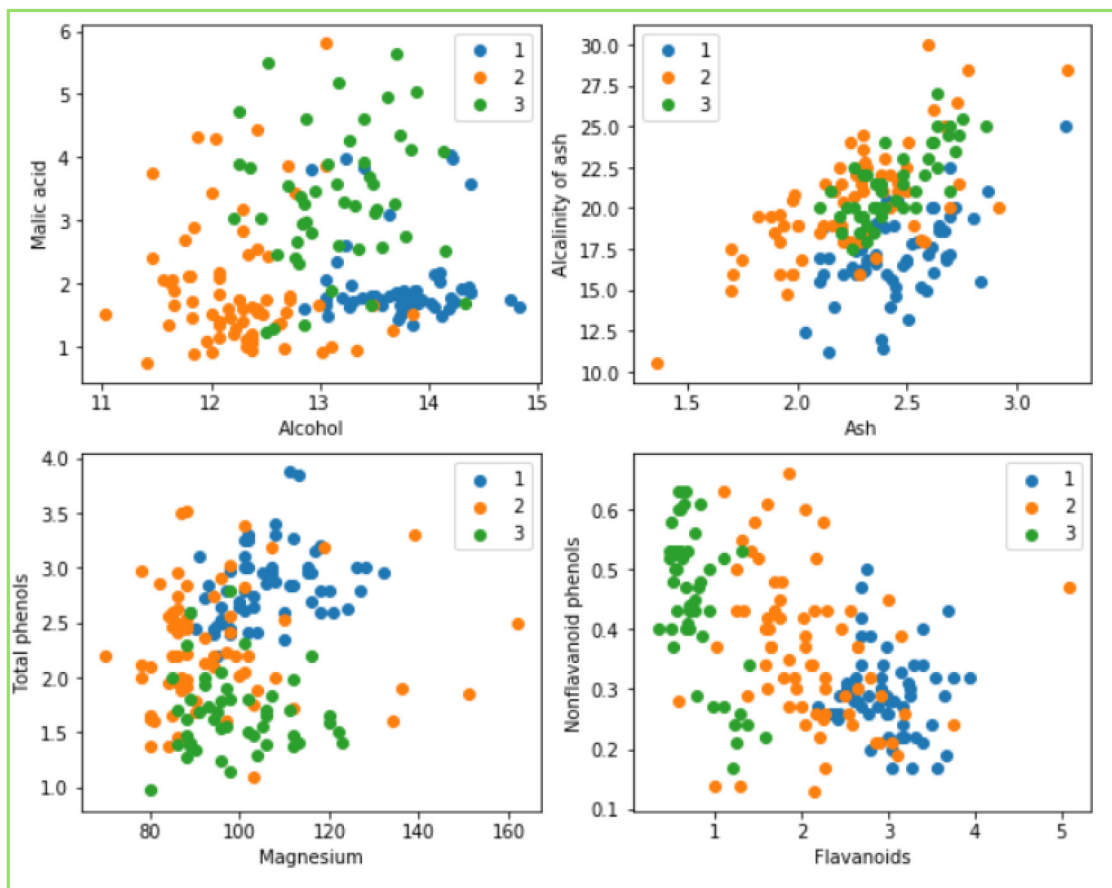


图1-1 查看数据分布

步骤 6 将数据集按 128:50 划分为训练集（已知类别样本）和验证集（待验证样本）：

```
train_idx = np.random.choice(178, 128, replace=False)
test_idx = np.array(list(set(range(178)) - set(train_idx)))
X_train, Y_train = X[train_idx], Y[train_idx]
X_test, Y_test = X[test_idx], Y[test_idx]
```

1.3.2 模型构建--计算距离

利用 MindSpore 提供的 tile, square, ReduceSum, sqrt, TopK 等算子，通过矩阵运算的方式同时计算输入样本 x 和已明确分类的其他样本 X_{train} 的距离，并计算出 top k 近邻。

```
class KnnNet(nn.Cell):
    def __init__(self, k):
        super(KnnNet, self).__init__()
        self.k = k

    def construct(self, x, X_train):
        # 平铺输入 x 以匹配 X_train 中的样本数
        x_tile = ops.tile(x, (128, 1))
        square_diff = ops.square(x_tile - X_train)
        square_dist = ops.sum(square_diff, 1)
```



```
dist = ops.sqrt(square_dist)
#-dist 表示值越大，样本就越接近
values, indices = ops.topk(-dist, self.k)
return indices

def knn(knn_net, x, X_train, Y_train):
    x, X_train = ms.Tensor(x), ms.Tensor(X_train)
    indices = knn_net(x, X_train)
    topk_cls = [0]*len(indices.asnumpy())
    for idx in indices.asnumpy():
        topk_cls[Y_train[idx]] += 1
    cls = np.argmax(topk_cls)
    return cls
```

1.3.3 模型预测

在验证集上验证 KNN 算法的有效性，取 $k=5$ ，验证精度接近 80%，说明 KNN 算法在该 3 分类任务上有效，能根据酒的 13 种属性判断出酒的品种。

```
acc = 0
knn_net = KnnNet(5)
for x, y in zip(X_test, Y_test):
    pred = knn(knn_net, x, X_train, Y_train)
    acc += (pred == y)
    print('label: %d, prediction: %s' % (y, pred))
print('Validation accuracy is %f' % (acc/len(Y_test)))
```

输出：

```
label: 1, prediction: 1
label: 3, prediction: 2
label: 3, prediction: 3
label: 1, prediction: 1
.....
Validation accuracy is 0.800000
```

1.4 实验小结

本实验使用 MindSpore 实现了 KNN 算法，用来解决 3 分类问题。取 wine 数据集上的 3 类样本，分为已知类别样本和待验证样本，从验证结果可以看出 KNN 算法在该任务上有效，能根据酒的 13 种属性判断出酒的品种。